HashiCorp Vault — Day 1: Foundations & Core Secrets Engines

Study material · Oct 16, 2026 · @Vivek Arora

Day 1 at a glance

By the end of Day 1, learners can explain why Vault exists, run it in dev and production mode, initialise and unseal it, drive it through CLI, API and UI, log in with Token, Userpass and LDAP, and store versioned secrets in KV v2.

Learning objectives

1. Describe secret sprawl and the six capabilities Vault uses to fix it.
2. Draw Vault's architecture: API, barrier, core, storage backend, and the key hierarchy.
3. Initialise a server with Shamir key shares and unseal it with a threshold of keys.
4. Choose correctly between dev mode and production mode, and write a production `vault.hcl`.
5. Perform the same operation through the CLI, the HTTP API (`curl`) and the web UI.
6. Enable and use Token, Userpass and LDAP auth methods, and attach policies to them.
7. Use KV v1 and KV v2, including versions, soft delete, undelete, destroy, metadata and check-and-set.

Suggested agenda

#	Module	Duration	Labs
1	Secrets management and why Vault	45 min	Lab 1.1
2	Architecture, seal/unseal, Shamir	75 min	Labs 1.3 (init/unseal) and 1.4 (Shamir)
3	Installation: dev vs production	60 min	Labs 1.2 and 1.3
4	CLI, API and UI walkthrough	45 min	Lab 1.5
5	Auth methods: Token, Userpass, LDAP	75 min	Labs 1.6 and 1.7
6	KV secrets engine v1 vs v2	60 min	Lab 1.8
7	Capstone lab	45 min	Lab 1.9

Lab prerequisites

- Linux VM (Ubuntu 22.04 or 24.04 recommended), 2 vCPU, 4 GB RAM, or any machine with Docker.
- Tools: `curl`, `jq`, `python3`, `pip`, and optionally `ldap-utils` for the LDAP lab.
- Ports 8200 (API/UI) and 8201 (cluster) free on the lab machine.
- Python client: `pip install hvac`.

Hands-on: every lab step, with expected-output screenshots, lives in the Vault Lab Guide — Day 1 & Day 2. Each module below ends with the lab that practises it, and each lab names its module, so the two guides stay in step. Day 3 has its own study guide and lab guide, linked from the Day 2 study guide.

Conventions: lines starting with `$` are shell commands; `VAULT_ADDR` and `VAULT_TOKEN` are set as shown in Lab 1.2. All passwords, tokens and keys shown are lab-only examples — never reuse them.

Module 1 — Secrets management and why Vault

A secret is anything that grants access; Vault puts every secret behind one identity check, one policy engine and one audit log, so secrets can be short-lived and revoked.

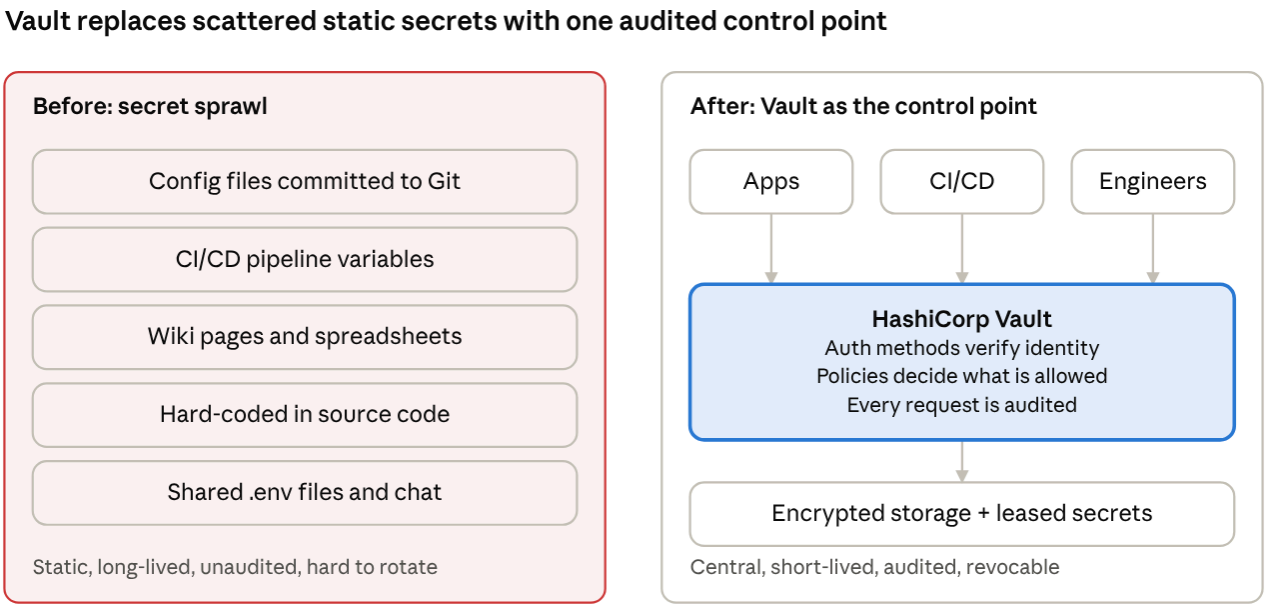
What counts as a secret

- Database usernames and passwords

- Cloud credentials (AWS access keys, Azure service principals, GCP service-account keys)
- API keys and tokens (Slack, GitHub, payment gateways)
- TLS certificates and private keys, SSH keys
- Encryption keys used by applications

The problem: secret sprawl

In most organisations secrets live in many places at once, are rarely rotated, and nobody can say who read them or when.



Secret sprawl vs Vault · before and after

The left side has no single place to rotate or revoke a leaked credential; the right side forces every consumer through authentication, policy and audit.

Vault's six core capabilities

Capability	What it means	Where in this course
Secure secret storage	Data is encrypted by Vault before it reaches the storage backend	Day 1 (KV)
Dynamic secrets	Credentials generated on demand, e.g. a new DB user per app instance	Day 2
Encryption as a service	Apps encrypt/decrypt data via API without holding keys (Transit)	Day 3 (Module 2)

Capability	What it means	Where in this course
Certificate authority (PKI)	Short-lived X.509 certificates issued on demand from a role	Day 3 (Module 1)
Leasing and renewal	Every dynamic secret has a TTL and must be renewed or it expires	Day 2
Revocation	Revoke one secret, or a whole tree (all secrets a user or app obtained)	Day 2
Audit	Every request and response is logged, with sensitive values HMAC-hashed	Day 1 (enable), Day 3 (Module 6)

The mental model for the whole course: **authenticate** (who are you?) → **authorise** (policy: what may you do?) → **access** (secrets engine does the work) → **audit** (record it).

Real-world use cases

Use case	Example	Vault feature
Application config secrets	A Spring Boot service reads its DB password at startup	KV v2 + AppRole or Kubernetes auth
CI/CD pipelines	Jenkins or GitHub Actions fetch deploy keys per run	JWT/OIDC auth + KV
Dynamic database credentials	Each pod gets its own PostgreSQL user that expires in 1 hour	Database secrets engine
Encrypting PII	Encrypt Aadhaar/PAN numbers before writing them to a database	Transit engine
Internal PKI	Issue 24-hour TLS certificates to microservices	PKI engine
Human access	Engineers log in with corporate Active Directory credentials	LDAP or OIDC auth

Editions: Vault Community (source-available, free), Vault Enterprise (namespaces, replication, HSM auto-unseal, Sentinel) and HCP Vault Dedicated (managed by HashiCorp). Everything on Day 1 works on Community.

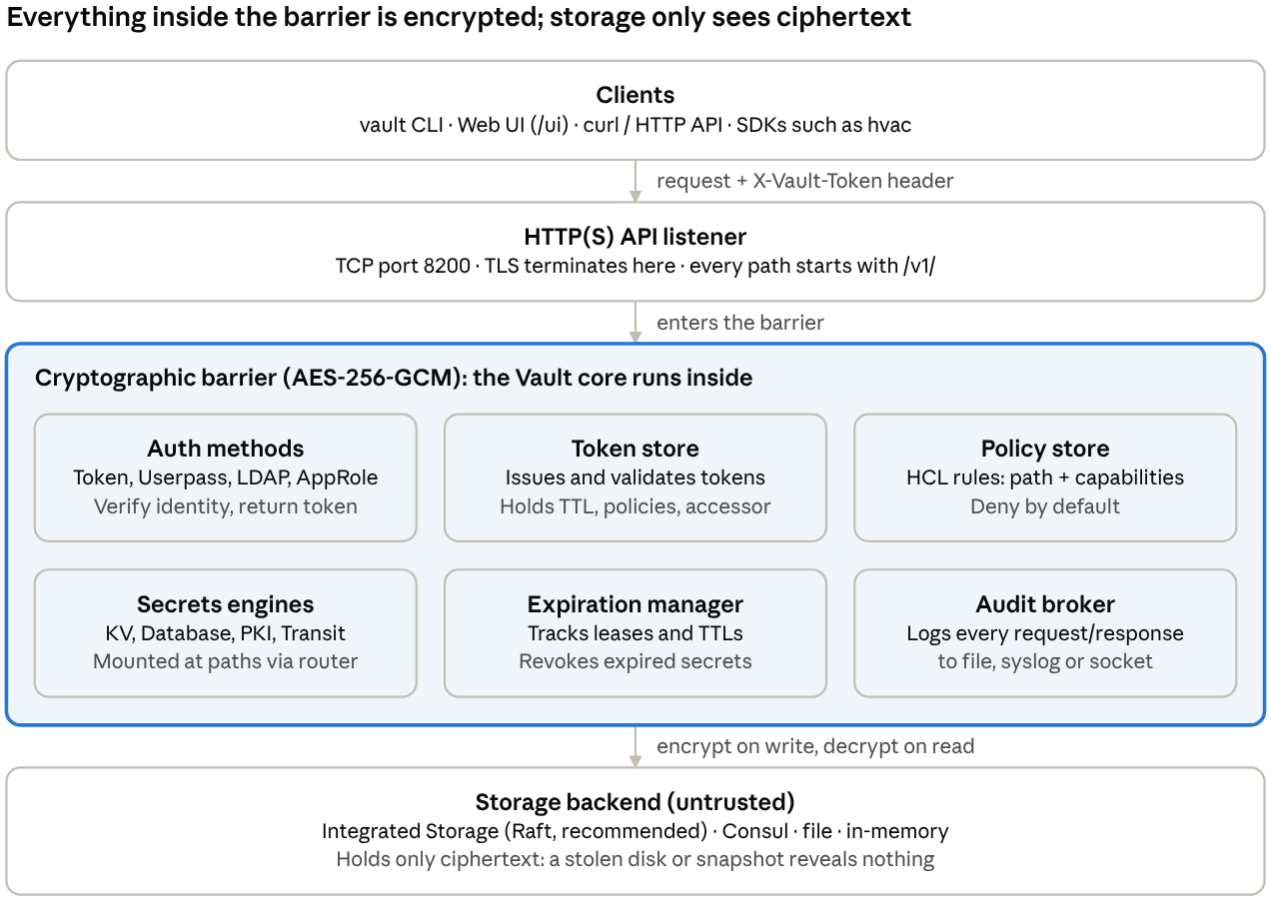
Hands-on

> **Do Lab 1.1 — Secret-sprawl audit** (10 min). Plant secrets in a sample repository, find them with grep and gitleaks, and decide how each would be rotated.

Module 2 — Vault architecture

Vault is a single binary whose core runs inside an encrypted barrier; the storage backend is treated as untrusted and only ever stores ciphertext.

Components



Vault architecture · clients, API, barrier, storage

Read it top-down: a request crosses the API, enters the barrier where identity and policy are checked, and anything written leaves the barrier encrypted.

What happens on a single request (e.g. `vault kv get secret/myapp`):

1. The client sends an HTTPS request to `/v1/secret/data/myapp` with header `X-Vault-Token`.

2. The token store validates the token and loads its attached policies.
3. The policy store checks the path against the policies; anything not explicitly allowed is denied.
4. The router forwards the request to the secrets engine mounted at `secret/`.
5. The engine reads the encrypted entry from storage; the barrier decrypts it in memory.
6. The audit broker logs the request and response (secret values HMAC-hashed), then the response returns.

Storage backend choices

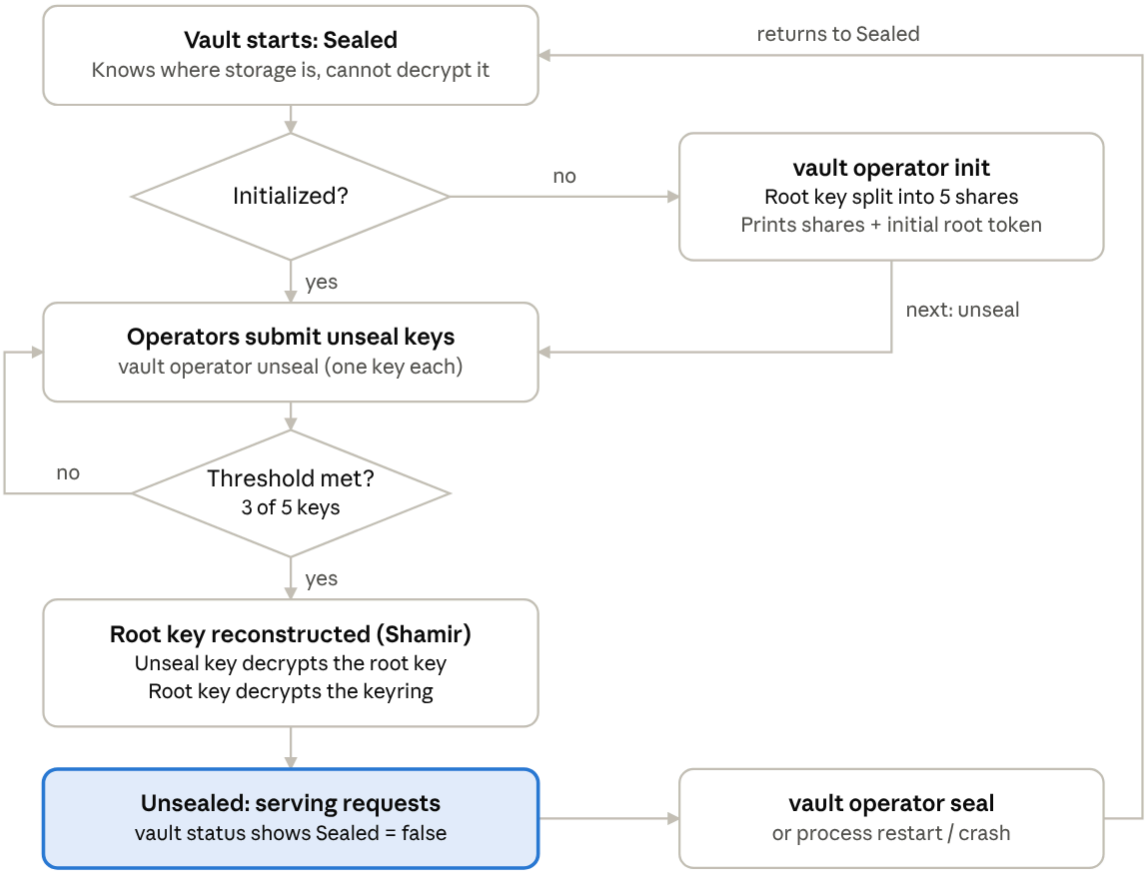
Backend	High availability	Typical use
Integrated Storage (Raft)	Yes, built in (3 or 5 nodes)	Recommended default for production
Consul	Yes, via a separate Consul cluster	Older deployments, existing Consul estates
File	No	Single-node labs and testing
In-memory	No, data lost on restart	Dev mode only

In an HA cluster one node is **active** and the others are **standby**; standbys forward client requests to the active node.

Seal and unseal

A sealed Vault knows where its data is but cannot decrypt it, so it answers only `sys/seal-status`, `sys/unseal` and a few health endpoints. Every restart, crash or manual seal returns it to this state.

Vault starts sealed; a threshold of key shares unseals it

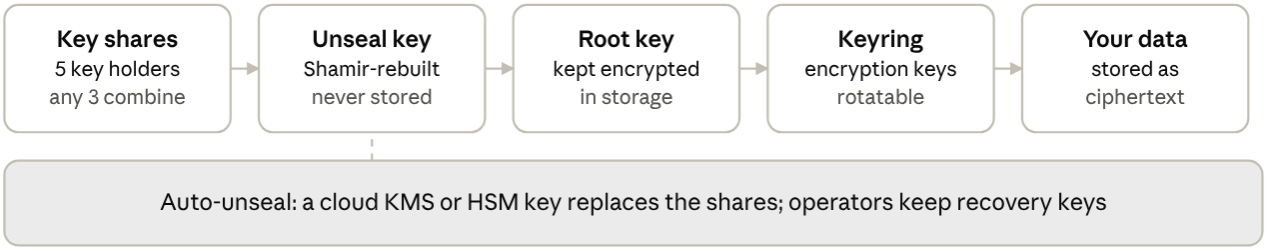


Seal / unseal flowchart · 2 decisions, 1 loop

Initialisation happens once per cluster; unsealing happens after every restart, and needs a quorum of key holders — no single person can open Vault alone.

Key hierarchy and Shamir's Secret Sharing

Unseal key, root key, keyring, data: each layer encrypts the next



Key hierarchy · 4 layers

Shamir's Secret Sharing hides the unseal key as the constant term of a random polynomial of degree $k - 1$. Each key holder gets one point on the curve. Any k points rebuild the polynomial (Lagrange interpolation); $k - 1$ points reveal nothing at all.

- Default at `vault operator init`: **5 shares, threshold 3** (`-key-shares`, `-key-threshold`).
- Distribute shares to different people; use `-pgp-keys` so each share is encrypted to its holder's PGP key.
- Rotate the shares or change the threshold with `vault operator rekey`; rotate the data encryption key with `vault operator rotate`.

> **Do Lab 1.4 — Shamir's Secret Sharing in Python** (10 min). A 25-line script splits a secret into 5 shares and shows that any 3 rebuild it while 2 reveal nothing.

Auto-unseal

	Shamir (default)	Auto-unseal
What protects the root key	Unseal key split into shares	Key in AWS KMS, Azure Key Vault, GCP Cloud KMS, Transit, or an HSM (Enterprise)
After a restart	Humans submit a quorum of keys	Vault unseals itself by calling the KMS
Keys given at init	Unseal keys	Recovery keys (for `generate-root` and rekey only; they cannot unseal)
Best for	Labs, air-gapped sites	Production, Kubernetes, autoscaling

```
# AWS KMS auto-unseal (add to vault.hcl)
seal "awskms" {
  region     = "ap-south-1"
  kms_key_id = "alias/vault-unseal"
}

# Azure Key Vault auto-unseal
seal "azurekeyvault" {
  tenant_id  = "<tenant-id>"
  vault_name = "kv-vault-unseal"
  key_name   = "vault-unseal-key"
}
```

Hands-on

> **Do Lab 1.3 — Production-style server: init, unseal, seal** (35 min). Initialise a Raft-backed server with 5 shares and a threshold of 3, unseal it, turn on auditing, create an admin, revoke the root token, and seal it again.

Key commands for reference:

```
$ vault status                                        # exit code 2 = sealed,
0 = unsealed
$ vault operator init -key-shares=5 -key-threshold=3
$ vault operator unseal                               # repeat until the
threshold is met
$ vault operator seal
$ vault operator key-status && vault operator rotate  # rotate the data
encryption key
$ vault operator rekey -init -key-shares=5 -key-threshold=3
$ vault operator generate-root -init                  # new root token from a
quorum of keys
$ curl -s -X PUT $VAULT_ADDR/v1/sys/unseal -d '{"key":"<unseal-key>"}'
```

Root token hygiene: use the initial root token only to set up auth methods and an admin policy, then revoke it (`vault token revoke <root-token>`). Regenerate one when needed with `vault operator generate-root`, which requires a quorum of unseal (or recovery) keys.

Module 3 — Installation and setup: dev mode vs production mode

Dev mode gives you an in-memory, already-unsealed server in one command for learning; production mode needs a config file, persistent storage, TLS, and an init/unseal ceremony.

Install the binary

```
# Ubuntu / Debian
$ wget -O- https://apt.releases.hashicorp.com/gpg | \
    sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
$ echo "deb [arch=$(dpkg --print-architecture) signed-
by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
    https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \
    sudo tee /etc/apt/sources.list.d/hashicorp.list
$ sudo apt update && sudo apt install -y vault

# RHEL / CentOS / Rocky
$ sudo yum install -y yum-utils
$ sudo yum-config-manager --add-repo
https://rpm.releases.hashicorp.com/RHEL/hashicorp.repo
$ sudo yum install -y vault

# macOS
$ brew tap hashicorp/tap && brew install hashicorp/tap/vault

# Verify and enable shell completion
$ vault version
$ vault -autocomplete-install && exec $SHELL
```

The Linux packages also create a `vault` system user, a systemd unit, `/etc/vault.d/vault.hcl`, the data directory `/opt/vault/data`, and a self-signed certificate in `/opt/vault/tls/`.

Dev mode vs production mode

Aspect	Dev mode	Production mode
Start command	`vault server -dev`	`vault server -config=/etc/vault.d/vault.hcl` (via systemd)
Storage	In-memory; everything lost on restart	Persistent: Integrated Storage (Raft) or Consul
TLS	Off (plain HTTP)	On; certificates required
Listener	`127.0.0.1:8200`	Whatever `listener "tcp"` sets, usually `0.0.0.0:8200`
Init / unseal	Automatic; one unseal key printed	Manual `vault operator init` + unseal, or auto-unseal

Aspect	Dev mode	Production mode
Root token	Printed; can be fixed with `-dev-root-token-id`	Printed once at init; revoke after setup
KV engine	KV v2 pre-mounted at `secret/`	Nothing mounted; you enable engines
Use it for	Learning, demos, local app development	Anything that holds real secrets

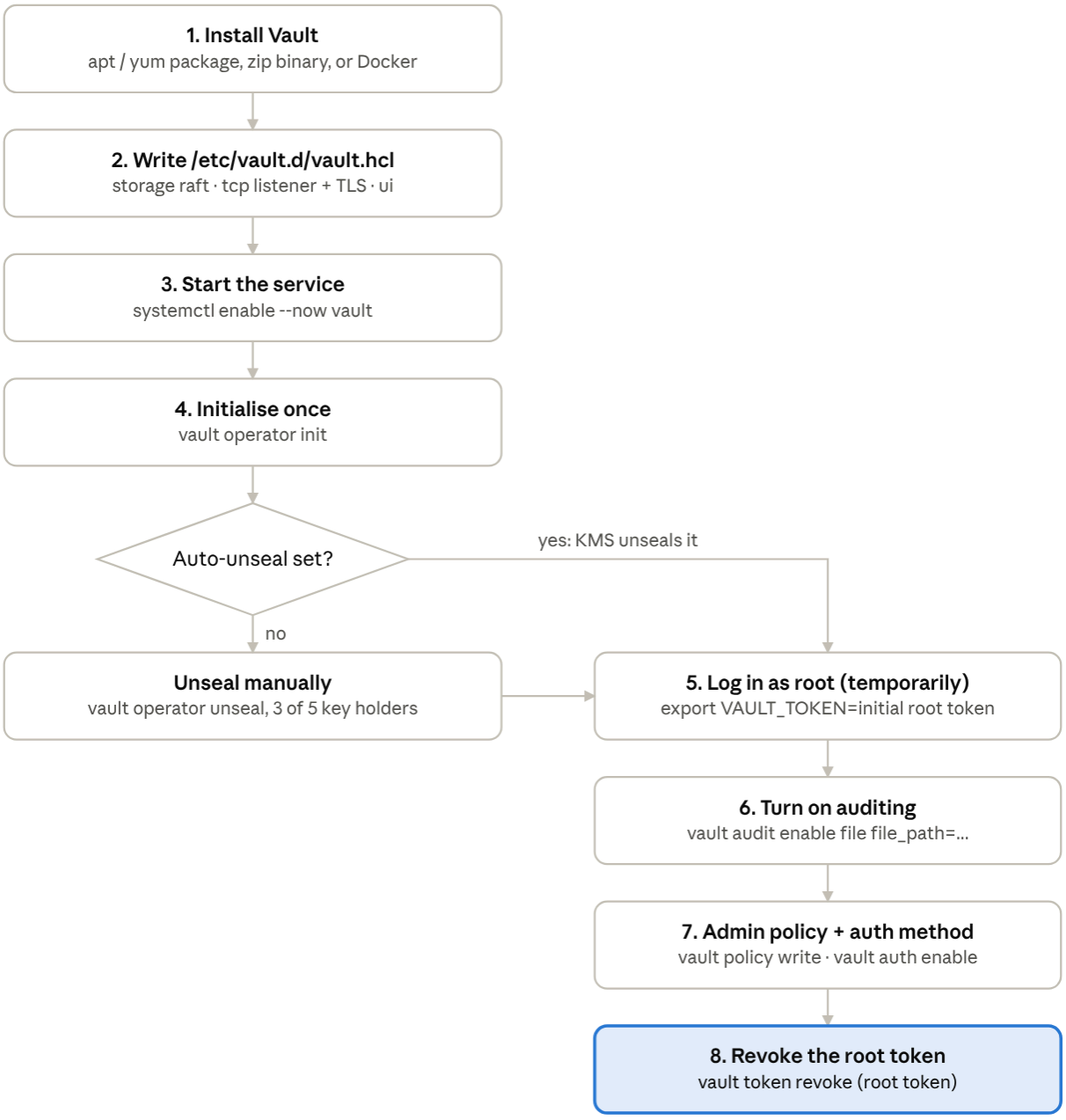
Never put real secrets in a dev server: it is unencrypted in transit, unsealed, and gone on restart.

Hands-on A — dev server

> **Do Lab 1.2 — Install Vault and run a dev server** (15 min). Install from the HashiCorp repository, start `vault server -dev`, write a first secret, and find it in the UI.

Hands-on B — production-style single node

A production server is ready only after the root token is revoked



Production bootstrap · 8 steps, 1 decision

Steps 1–4 run once per cluster; the unseal branch repeats after every restart, and step 8 is what separates a lab from a production setup.

Step 2 — configuration file

```
# /etc/vault.d/vault.hcl
ui            = true
disable_mlock = true                      # recommended with Integrated
Storage
api_addr      = "https://vault-1.lab.local:8200"
cluster_addr  = "https://vault-1.lab.local:8201"
log_level     = "info"

storage "raft" {
  path    = "/opt/vault/data"
  node_id = "vault-1"
}

listener "tcp" {
  address       = "0.0.0.0:8200"
  tls_cert_file = "/opt/vault/tls/tls.crt"
  tls_key_file  = "/opt/vault/tls/tls.key"
}
```

For a classroom VM without certificates, swap the listener for this and use `http://` in `api_addr` only; `cluster_addr` stays `https://` because Vault always secures cluster traffic itself (lab only):

```
listener "tcp" {
  address     = "0.0.0.0:8200"
  tls_disable = "true"
}
```

Stanza / key	Purpose
`storage`	Where encrypted data lives (`raft` , `consul` , `file`)
`listener "tcp"`	Address, port and TLS for the API and UI
`seal`	Optional auto-unseal (`awskms` , `azurekeyvault` , `gcpckms` , `transit`)
`api_addr`	Address other nodes and clients are redirected to
`cluster_addr`	Node-to-node address (port 8201) for Raft and request forwarding
`ui`	Enables the web UI at `/ui`
`disable_mlock`	Skips memory locking; set `true` with Raft storage

Stanza / key	Purpose
`telemetry`	Metrics for Prometheus/StatsD (Day 3, Module 6)

> **Do Lab 1.3** for steps 3–8 of the bootstrap — start the service, initialise, unseal, enable audit, create an admin, revoke root — each with expected output.

systemd unit (only needed for a zip-binary install; the package ships one):

```
# /etc/systemd/system/vault.service
[Unit]
Description=HashiCorp Vault
Requires=network-online.target
After=network-online.target
ConditionFileNotEmpty=/etc/vault.d/vault.hcl

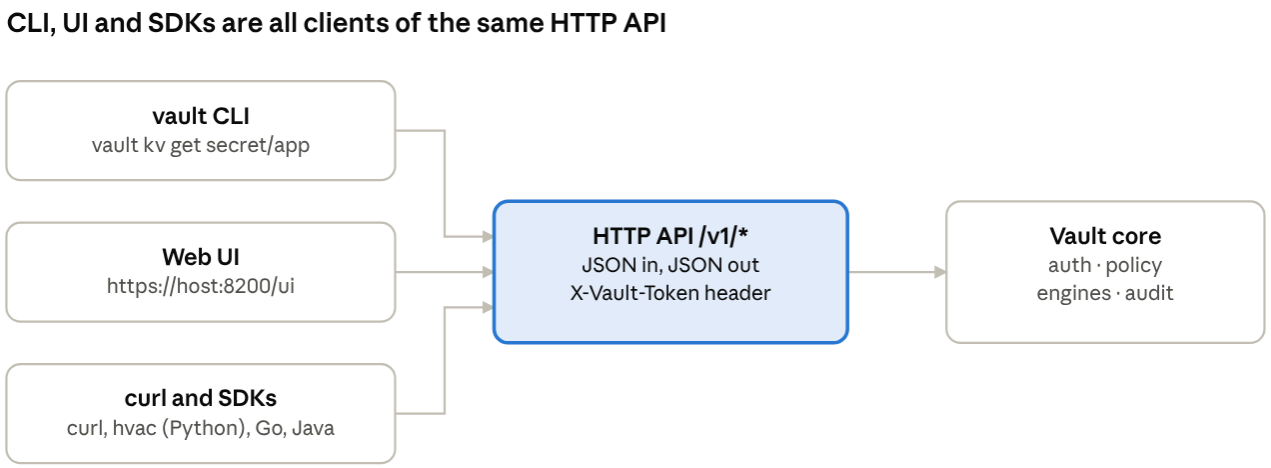
[Service]
User=vault
Group=vault
ExecStart=/usr/bin/vault server -config=/etc/vault.d/vault.hcl
ExecReload=/bin/kill --signal HUP $MAINPID
CapabilityBoundingSet=CAP_SYSLOG CAP_IPC_LOCK
AmbientCapabilities=CAP_IPC_LOCK
LimitNOFILE=65536
LimitMEMLOCK=infinity
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Three-node Raft clusters add a `retry_join { leader_api_addr = "https://vault-1:8200" }` block inside `storage "raft"`; clustering, failover and snapshots are covered on Day 3 (Module 5, Lab 3.5).

Module 4 — Vault CLI, API and UI walkthrough

The CLI and UI are thin clients of the HTTP API, so anything you do in one you can do in the others: learn the CLI, read the API, demo the UI.



Three interfaces, one API

The practical consequence: `vault <cmd> -output-curl-string` shows you the exact API call behind any CLI command.

Environment variables

Variable	Purpose	Example
`VAULT_ADDR`	Server address	`https://vault.lab.local:8200`
`VAULT_TOKEN`	Token used for requests (overrides `~/.vault-token`)	`hvs.CAESI...`
`VAULT_CACERT`	CA certificate to trust	`/opt/vault/tls/ca.crt`
`VAULT_SKIP_VERIFY`	Skip TLS verification (lab only)	`true`
`VAULT_FORMAT`	Default output format	`json`, `table`, `yaml`
`VAULT_NAMESPACE`	Target namespace (Enterprise / HCP)	`admin/team-a`

After `vault login`, the CLI's token helper saves the token in `~/.vault-token`; `VAULT_TOKEN`, if set, takes priority.

CLI essentials

```
$ vault -h                          # top-level commands
$ vault kv -h                       # help for a sub-command
$ vault status
$ vault login                       # prompts for a token
$ vault token lookup                # who am I, which policies, TTL left
$ vault secrets list -detailed      # mounted secrets engines
$ vault auth list                   # enabled auth methods
$ vault path-help secret/           # built-in docs for any path

# Output formats and field extraction
$ vault kv get -format=json secret/hello | jq -r '.data.data.msg'
$ vault kv get -field=msg secret/hello

# Learn the API from the CLI
$ vault kv get -output-curl-string secret/hello
$ vault kv get -output-policy secret/hello      # the policy this call needs
```

Generic verbs work on any path: `vault read`, `vault write`, `vault list`, `vault delete`. `vault kv ...` is a helper that adds KV v2 path handling on top.

HTTP API essentials

Every endpoint lives under `/v1/`. Authenticate with `X-Vault-Token: <token>` (or `Authorization: Bearer <token>`); `LIST` is sent as `-X LIST` or `GET ...?list=true`.

Action	Method + path
Health check	`GET /v1/sys/health`
Seal status	`GET /v1/sys/seal-status`
Who am I	`GET /v1/auth/token/lookup-self`
Log in with userpass	`POST /v1/auth/userpass/login/<username>`
Write a KV v2 secret	`POST /v1/secret/data/<path>`
Read a KV v2 secret	`GET /v1/secret/data/<path>`
List keys	`LIST /v1/secret/metadata/<folder>`

```
# Health (no token needed)
$ curl -s $VAULT_ADDR/v1/sys/health | jq

# Write: KV v2 wants the payload wrapped in {"data": {...}}
$ curl -s -H "X-Vault-Token: $VAULT_TOKEN" -X POST \
    -d '{"data":{"username":"app","password":"s3cr3t"}}' \
    $VAULT_ADDR/v1/secret/data/myapp | jq

# Read
$ curl -s -H "X-Vault-Token: $VAULT_TOKEN" \
    $VAULT_ADDR/v1/secret/data/myapp | jq '.data.data'

# List
$ curl -s -H "X-Vault-Token: $VAULT_TOKEN" -X LIST \
    $VAULT_ADDR/v1/secret/metadata | jq '.data.keys'

# Log in and capture a token
$ TOKEN=$(curl -s -X POST -d '{"password":"Passw0rd!"}' \
    $VAULT_ADDR/v1/auth/userpass/login/alice | jq -r '.auth.client_token')
```

`/v1/sys/health` status codes are what load balancers check:

HTTP code	Meaning
200	Initialised, unsealed, active
429	Unsealed standby
473	Performance standby (Enterprise)
472	Disaster-recovery secondary (Enterprise)
501	Not initialised
503	Sealed

UI walkthrough

1. Browse to `$VAULT_ADDR/ui` and sign in with method **Token** (or Username for userpass, LDAP for LDAP).
2. **Secrets Engines**: see `secret/`, click **Enable new engine** to mount KV, PKI, Transit and others.
3. Open `secret/` → **Create secret**: enter a path and key/value pairs; open **Version history** to see versions.

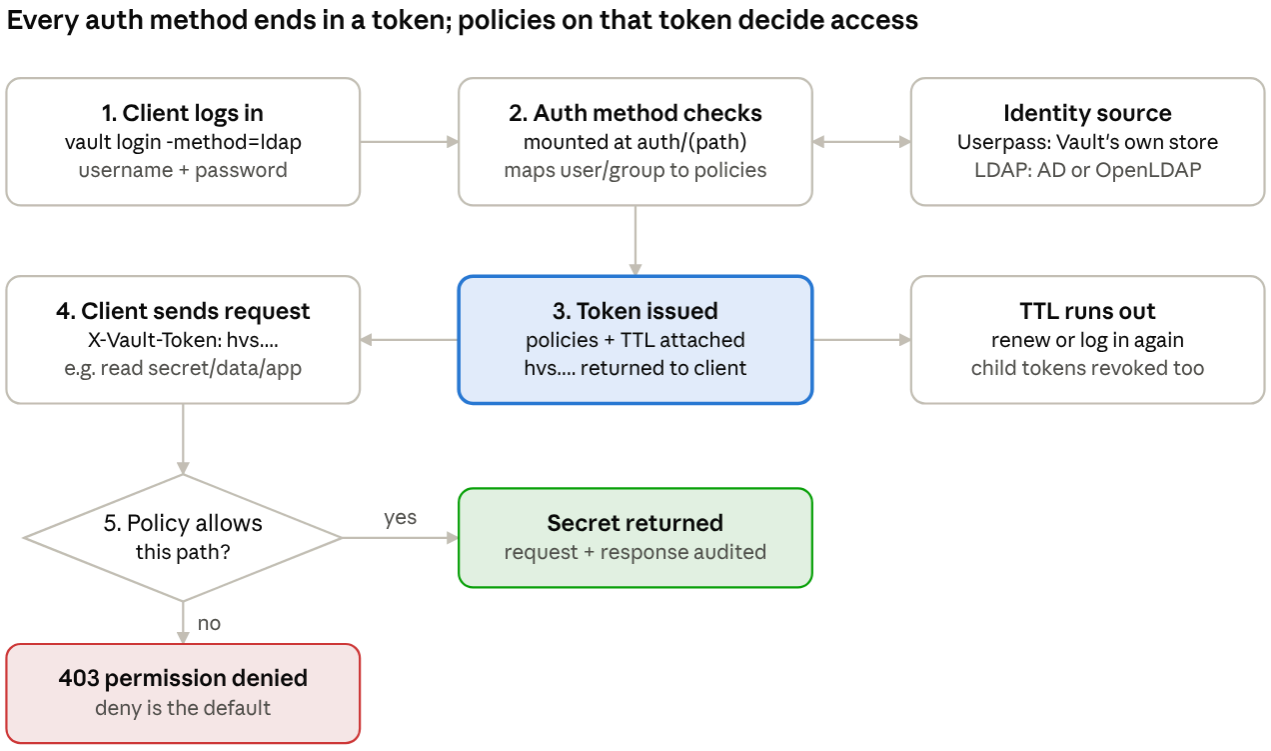
4. **Access → Authentication Methods**: enable and configure userpass or LDAP; manage users and groups.
5. **Policies**: create and edit ACL policies in an HCL editor.
6. **Tools**: wrap, unwrap, lookup, rewrap, random bytes, and hash.
7. The terminal icon opens the **Web REPL**, a browser CLI that accepts `read`, `write`, `list` and `delete`.

Hands-on

> **Do Lab 1.5 — CLI, API and UI: the same task three ways** (15 min). Write and read one secret through the CLI, `curl` and the UI, and compare each CLI command with its `-output-curl-string`.

Module 5 — Authentication methods: Token, Userpass, LDAP

Every login, whatever the method, ends in the same thing — a token carrying policies and a TTL; auth methods differ only in how they verify who you are.



Login to access · 5 steps, 1 decision

Steps 1–3 happen once per session; steps 4–5 happen on every request, which is why a token's policies, not the login method, decide what a user can touch.

Method	Mount path	Verifies identity against	Best for
Token	`auth/token` (always on, cannot be disabled)	Vault's token store	Bootstrapping, automation, child tokens
Userpass	`auth/userpass`	Usernames and bcrypt-hashed passwords stored in Vault	Labs, small teams, break-glass accounts
LDAP	`auth/ldap`	Active Directory or OpenLDAP	Enterprise users with existing AD groups
Later days	AppRole, Kubernetes, OIDC, JWT, AWS, Azure	Machine or cloud identity	Applications and pipelines

Policy primer (needed for this module)

Policies are HCL rules of *path + capabilities*. Vault denies anything not explicitly allowed; `deny` beats every grant. Capabilities: `create`, `read`, `update`, `patch`, `delete`, `list`, `sudo`, `deny`.

```
# dev-read.hcl — read-only access to one app's secrets (KV v2 at secret/)
path "secret/data/myapp/*" {
  capabilities = ["read"]
}
path "secret/metadata/myapp/*" {
  capabilities = ["list"]
}
```

```
# admin.hcl — lab administrator used in Module 3, step 7 (tighten for
production)
path "sys/*" {
  capabilities = ["create", "read", "update", "delete", "list", "sudo"]
}
path "auth/*" {
  capabilities = ["create", "read", "update", "delete", "list", "sudo"]
}
path "secret/*" {
  capabilities = ["create", "read", "update", "patch", "delete", "list"]
}
path "identity/*" {
  capabilities = ["create", "read", "update", "delete", "list"]
}
```

```
$ vault policy write dev-read dev-read.hcl
$ vault policy write admin admin.hcl
$ vault policy list                # default and root always exist
$ vault policy read dev-read
```

Token auth

Concept	What to know
Root token	Policy `root`, can do anything, no TTL; revoke after setup
Service token (`hvs.`)	Default type; renewable, can create children, persisted in storage
Batch token (`hvb.`)	Lightweight, not persisted, not renewable, no children; for high-volume short jobs
TTL / max TTL	TTL is the current lifetime; renewals cannot extend past max TTL
Periodic token	No max TTL; lives as long as it is renewed within each period
Accessor	A reference to a token that can look up, renew or revoke it without revealing the token
Token hierarchy	Revoking a parent revokes all its children; `-orphan` creates a token with no parent

```
# Create tokens
$ vault token create -policy=dev-read -ttl=1h
$ vault token create -policy=dev-read -ttl=1h -explicit-max-ttl=4h
$ vault token create -type=batch -policy=dev-read -ttl=20m
$ vault token create -policy=dev-read -period=24h      # periodic

# Inspect and use
$ vault token lookup <token>
$ vault token lookup -accessor <accessor>
$ vault token capabilities <token> secret/data/myapp/config
$ vault login <token>

# Renew and revoke
$ vault token renew <token>
$ vault token revoke <token>
$ vault token revoke -accessor <accessor>
```

> **Do Lab 1.6 — Tokens, policies and Userpass** (25 min). Issue scoped tokens, check capabilities, watch a child token die with its parent, and log in as a Userpass user.

Userpass

```
# Enable and tune
$ vault auth enable userpass
$ vault auth tune -default-lease-ttl=1h -max-lease-ttl=8h userpass/

# Create a user bound to a policy
$ vault write auth/userpass/users/alice \
    password='Passw0rd!' \
    token_policies=dev-read \
    token_ttl=1h token_max_ttl=4h
$ vault list auth/userpass/users
$ vault read auth/userpass/users/alice

# Log in as alice (prompts for the password)
$ vault login -method=userpass username=alice
$ vault kv get secret/myapp/config               # allowed
$ vault kv put secret/myapp/config x=y          # 403 permission denied

# Admin tasks
$ vault write auth/userpass/users/alice/password password='N3wPass!'
$ vault write auth/userpass/users/alice/policies token_policies=dev-read,dev-
write
$ vault delete auth/userpass/users/alice
```

The same method can be mounted twice at different paths, e.g. `vault auth enable -path=breakglass userpass`, then `vault login -method=userpass -path=breakglass username=...`.

LDAP

> **Do Lab 1.7 — LDAP auth with OpenLDAP** (25 min). An OpenLDAP container with user `bob` in group `devops`, the configuration below, and group-to-policy mapping.

Configure Vault (if Vault itself runs in Docker, replace `localhost` with the host's IP or a shared Docker network name):

```
$ vault auth enable ldap

$ vault write auth/ldap/config \
    url="ldap://localhost:389" \
    binddn="cn=admin,dc=lab,dc=local" \
    bindpass="admin" \
    userdn="ou=users,dc=lab,dc=local" \
    userattr="uid" \
    groupdn="ou=groups,dc=lab,dc=local" \
    groupfilter="(&(objectClass=groupOfNames)(member={{.UserDN}}))" \
    groupattr="cn" \
    starttls=false insecure_tls=true          # lab only; use ldaps:// in
production

# Map the LDAP group to a Vault policy
$ vault write auth/ldap/groups/devops policies=dev-read

# Optional: map an individual user too
$ vault write auth/ldap/users/bob policies=default

# Log in as bob
$ vault login -method=ldap username=bob     # password BobPass123
$ vault token lookup                        # policies: default, dev-read
```

Active Directory equivalents:

```
$ vault write auth/ldap/config \
    url="ldaps://dc01.corp.local" \
    binddn="CN=svc-vault,OU=Service Accounts,DC=corp,DC=local" \
    bindpass="<from a secure source>" \
    userdn="OU=Users,DC=corp,DC=local" \
    userattr="sAMAccountName" \
    upndomain="corp.local" \
    groupdn="OU=Groups,DC=corp,DC=local" \
    groupfilter="(&(objectClass=group)(member:1.2.840.113556.1.4.1941:=
{{.UserDN}}))" \
    groupattr="cn" \
    certificate=@/etc/ssl/certs/corp-ca.pem
```

The OID `1.2.840.113556.1.4.1941` makes AD resolve nested group membership.

Which method when

Question	Token	Userpass	LDAP
Who manages the credentials?	Vault	Vault admins	Your directory team
Password policy and lockout	Not applicable	Vault only	Enforced by AD
Onboarding / offboarding	Manual	Manual per user	Automatic via AD groups
Typical users	Automation, bootstrap	Lab users, break-glass	Employees

Module 6 — KV secrets engine: v1 vs v2 and versioning

KV v1 keeps one value per path with no history; KV v2 keeps versions (10 by default), adds soft delete, undelete, destroy, metadata and check-and-set. Use v2 unless you have a specific reason not to.

v1 vs v2 at a glance

Feature	KV v1	KV v2
Enable	`vault secrets enable -version=1 kv`	`vault secrets enable kv-v2` (or `-version=2 kv`)

Feature	KV v1	KV v2
Write behaviour	Overwrites; old value lost	Creates a new version; old versions kept
Delete	Permanent	Soft delete; `undelete` restores, `destroy` wipes
Metadata	None	Created/updated times, versions, custom metadata
Check-and-set	No	Yes (`-cas` , `cas_required`)
API data path	`/v1/<mount>/<path>`	`/v1/<mount>/data/<path>` (+ `metadata/` , `delete/` , `undelete/` , `destroy/`)
API write payload	`{"key":"value"}`	`{"data":{"key":"value"}}`
Storage and speed	Slightly smaller and faster	Extra storage for versions and metadata
Dev server default	—	`secret/` is KV v2

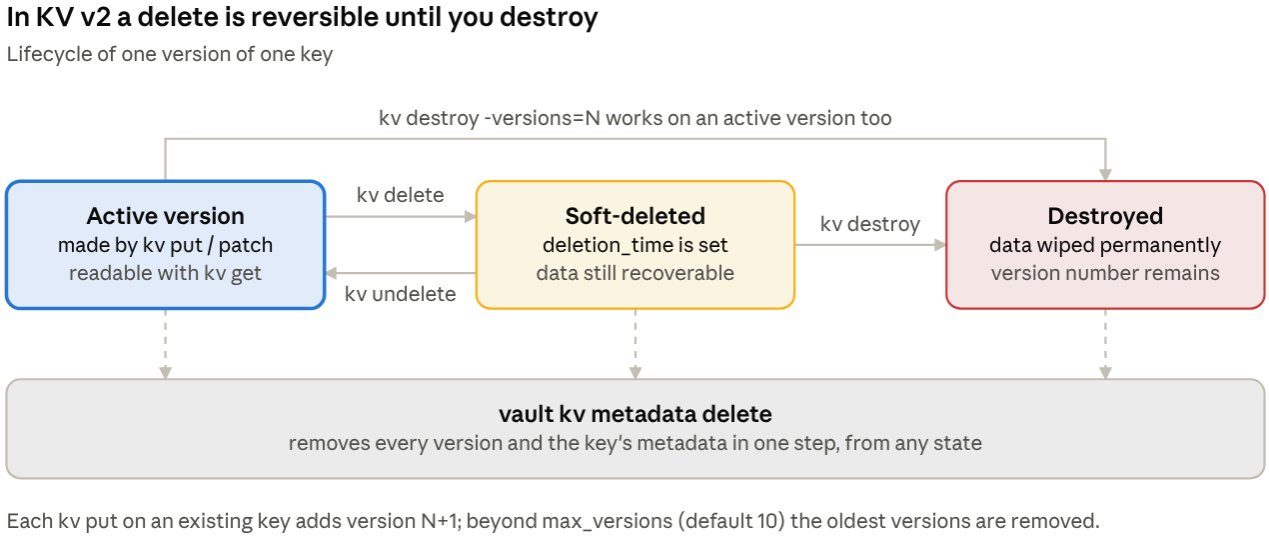
How KV v2 paths map

The `vault kv` CLI hides the extra path segment; the API and **policies** do not. The most common Day 1 mistake is writing a policy for `secret/myapp/*` on a v2 mount — it must be `secret/data/myapp/*`.

CLI	API path	Policy path needed
`vault kv put / get secret/myapp`	`secret/data/myapp`	`secret/data/myapp`
`vault kv list secret/`	`secret/metadata/`	`secret/metadata/*` with `list`
`vault kv metadata get secret/myapp`	`secret/metadata/myapp`	`secret/metadata/myapp`
`vault kv delete secret/myapp`	`secret/data/myapp` (DELETE) or `secret/delete/myapp`	`delete` on data path, or `update` on `delete/`

CLI	API path	Policy path needed
`vault kv undelete -versions=2 secret/myapp`	`secret/undelete/myapp`	`secret/undelete/myapp`
`vault kv destroy -versions=1 secret/myapp`	`secret/destroy/myapp`	`secret/destroy/myapp`

Version lifecycle



KV v2 version lifecycle · 3 states

A soft delete only hides data, so an accidental `kv delete` is one `undelete` away from recovery; `destroy` and `metadata delete` are the irreversible operations to restrict in policies.

Hands-on

> **Do Lab 1.8 — KV v1 vs v2 and versioning** (25 min). Compare v1 and v2, then take one secret through versions, soft delete, undelete, destroy, rollback and check-and-set.

Keep secrets out of shell history:

```
$ vault kv put -mount=kv2 app/db password=-               # reads the value
from stdin
$ vault kv put -mount=kv2 app/tls cert=@server.crt key=@server.key   # values
from files
$ vault kv put -mount=kv2 app/json @payload.json          # whole secret from
a JSON file
```

Upgrade a v1 mount to v2 (the mount is briefly unavailable while it upgrades):

```
$ vault kv enable-versioning kv1/
```

KV v2 through the API

```
H="X-Vault-Token: $VAULT_TOKEN"

# Create only if absent (cas=0)
$ curl -s -H "$H" -X POST -d '{"options":{"cas":0},"data":
{"api_key":"abc123"}}' \
    $VAULT_ADDR/v1/kv2/data/app/api | jq '.data.version'

# Read a specific version
$ curl -s -H "$H" "$VAULT_ADDR/v1/kv2/data/app/api?version=1" | jq
'.data.data'

# Soft delete, undelete, destroy specific versions
$ curl -s -H "$H" -X POST -d '{"versions":[1]}'
$VAULT_ADDR/v1/kv2/delete/app/api
$ curl -s -H "$H" -X POST -d '{"versions":[1]}'
$VAULT_ADDR/v1/kv2/undelete/app/api
$ curl -s -H "$H" -X POST -d '{"versions":[1]}'
$VAULT_ADDR/v1/kv2/destroy/app/api

# Metadata and listing
$ curl -s -H "$H" $VAULT_ADDR/v1/kv2/metadata/app/api | jq '.data.versions'
$ curl -s -H "$H" -X LIST $VAULT_ADDR/v1/kv2/metadata/app | jq '.data.keys'
```

KV v2 from Python (hvac)

```
# kv_demo.py — pip install hvac
import os
import hvac

client = hvac.Client(url=os.environ["VAULT_ADDR"],
token=os.environ["VAULT_TOKEN"])
```

```
assert client.is_authenticated(), "Vault token is invalid or expired"

kv = client.secrets.kv.v2
MOUNT, PATH = "kv2", "app/py"

# Two writes -> versions 1 and 2
kv.create_or_update_secret(mount_point=MOUNT, path=PATH, secret={"api_key":
"abc123"})
kv.create_or_update_secret(mount_point=MOUNT, path=PATH, secret={"api_key":
"xyz789"})

latest = kv.read_secret_version(mount_point=MOUNT, path=PATH,
raise_on_deleted_version=True)
print("latest:", latest["data"]["data"], "version", latest["data"]
["metadata"]["version"])

v1 = kv.read_secret_version(mount_point=MOUNT, path=PATH, version=1,
raise_on_deleted_version=True)
print("v1:", v1["data"]["data"])

meta = kv.read_secret_metadata(mount_point=MOUNT, path=PATH)
print("current_version:", meta["data"]["current_version"])

# Lifecycle operations
kv.delete_latest_version_of_secret(mount_point=MOUNT, path=PATH)        #
soft delete v2
kv.undelete_secret_versions(mount_point=MOUNT, path=PATH, versions=[2])  #
restore v2
kv.destroy_secret_versions(mount_point=MOUNT, path=PATH, versions=[1])   #
wipe v1
```

Common mistakes

Symptom	Cause	Fix
`403 permission denied` on a path the policy seems to allow	Policy written without `data/` on a KV v2 mount	Use `secret/data/...` and `secret/metadata/...`
`vault kv list` returns 403	No `list` on the metadata path	Add `list` on `secret/metadata/...`
`kv patch` fails with 403	Policy has `update` but not `patch`	Add the `patch` capability

Symptom	Cause	Fix
Empty data with a `deletion_time`	The version was soft-deleted	`vault kv undelete -versions=N`
`check-and-set parameter did not match`	`-cas` value is not the current version	Read metadata, retry with the current version
404 or odd results when using `data/`	Calling v2-style paths on a KV v1 mount	Check `vault secrets list -detailed` for the version option

Capstone lab — Day 1 end to end

> **Do Lab 1.9 — Day 1 capstone** (45 min). One script builds a production-style Vault from nothing — Raft storage, Shamir unseal, audit, KV v2, policies, Userpass — and a Python app then reads its config with least privilege. The script, the app, expected output and the self-check list are in the Lab Guide.

Troubleshooting, cheat sheet and review

Most Day 1 errors come from four things: the wrong `VAULT_ADDR`, a sealed server, a missing or expired token, and a policy path that does not match the API path.

Troubleshooting

Error message (abridged)	Likely cause	Fix
`connect: connection refused`	Server not running, or wrong address/port	`systemctl status vault`, `journalctl -u vault -f`, check `VAULT_ADDR`
`server gave HTTP response to HTTPS client`	`VAULT_ADDR` is `https://` but the listener has `tls_disable`	Use `http://` (lab) or configure TLS
`x509: certificate signed by unknown authority`	CLI does not trust the server certificate	`export VAULT_CACERT=<ca.pem>`; `VAULT_SKIP_VERIFY=true` in labs only
`Code: 501 ... not initialized`	Fresh server	`vault operator init`

Error message (abridged)	Likely cause	Fix
`Code: 503 ... Vault is sealed`	Restart, crash or manual seal	`vault operator unseal` x threshold, or check the auto-unseal KMS
`missing client token`	No `VAULT_TOKEN` and no `~/.vault-token`	`vault login` or export `VAULT_TOKEN`
`Code: 403 ... permission denied`	Policy does not cover the path, or token expired	`vault token lookup`, `vault token capabilities <path>`, `-output-policy`
`failed to lock memory` / mlock error	No `IPC_LOCK` capability	`disable_mlock = true` (Raft) or grant `CAP_IPC_LOCK`
`permission denied` opening `audit.log`	Log directory not owned by the `vault` user	`chown vault:vault /var/log/vault`
`ldap operation failed: failed to bind`	Wrong `binddn`, `bindpass`, `userdn` or `userattr`	Test the same values with `ldapsearch` first

Diagnostic commands:

```
$ vault status                          # sealed? HA mode? storage type?
$ vault monitor -log-level=debug        # stream the server's logs through the
API
$ vault token lookup                    # policies and TTL left on your token
$ vault token capabilities secret/data/myapp/config
$ vault read sys/mounts                 # what is mounted where (or: vault
secrets list -detailed)
$ journalctl -u vault --since "10 min ago"
```

Cheat sheet

Task	Command
Start a dev server	`vault server -dev -dev-root-token-id=root`
Check status	`vault status`
Initialise	`vault operator init -key-shares=5 -key-threshold=3`
Unseal / seal	`vault operator unseal` · `vault operator seal`

Task	Command
Rotate data key / rekey shares	`vault operator rotate` · `vault operator rekey -init`
Log in	`vault login` · `vault login -method=userpass username=alice` · `-method=ldap`
Create / revoke a token	`vault token create -policy=dev-read -ttl=1h` · `vault token revoke <t>`
Enable an auth method	`vault auth enable userpass` · `vault auth enable ldap`
Write a policy	`vault policy write dev-read dev-read.hcl`
Enable KV	`vault secrets enable -version=1 kv` · `vault secrets enable kv-v2`
Write / read / patch	`vault kv put` · `vault kv get [-version=N]` · `vault kv patch`
Delete / undelete / destroy	`vault kv delete` · `vault kv undelete -versions=N` · `vault kv destroy -versions=N`
Metadata	`vault kv metadata get` · `vault kv metadata put -max-versions=5` · `vault kv metadata delete`
Enable audit	`vault audit enable file file_path=/var/log/vault/audit.log`
See the API call behind a command	`<command> -output-curl-string`

Review quiz

1. Where does Vault encrypt data, and what does the storage backend see?
2. What are the default key shares and threshold at `vault operator init`?
3. A Vault server restarts. What state is it in, and what brings it back?
4. With auto-unseal configured, what does `vault operator init` hand out instead of unseal keys, and can they unseal Vault?
5. Name three ways dev mode differs from production mode.
6. Which HTTP header carries the token, and what code does `/v1/sys/health` return when Vault is sealed?
7. A policy grants `read` on `secret/myapp/*` but reads of `vault kv get secret/myapp/db` fail with 403 on a KV v2 mount. Why?

8. What happens to child tokens when their parent is revoked?
9. What is the difference between `vault kv delete` and `vault kv destroy`?
10. What does `-cas=0` do on a KV v2 write?
11. How does an LDAP user end up with Vault policies?
12. Why should the initial root token be revoked after setup, and how do you get one back?

Answers

1. At the barrier, inside Vault, with AES-256-GCM; storage only ever sees ciphertext.
2. 5 shares, threshold 3.
3. Sealed. A quorum of unseal keys (`vault operator unseal`), or auto-unseal via the configured KMS/HSM.
4. Recovery keys. No — they authorise operations such as `generate-root` and rekey, but cannot unseal.
5. Any three of: in-memory storage, auto-unsealed, HTTP only, root token printed, KV v2 pre-mounted at `secret/`, listens on 127.0.0.1.
6. `X-Vault-Token` (or `Authorization: Bearer`); 503.
7. KV v2 API paths include `data/`; the policy needs `secret/data/myapp/*`.
8. They are revoked too, unless they were created as orphans.
9. Delete is a soft delete that `undelete` can reverse; destroy wipes the version's data permanently.
10. The write succeeds only if the key does not exist yet.
11. Vault maps their LDAP groups (`auth/ldap/groups/<group>`) and optionally their username (`auth/ldap/users/<user>`) to policies; the `default` policy is added automatically.
12. It has unlimited power and no TTL, so a leak is catastrophic; regenerate one with `vault operator generate-root` and a quorum of unseal or recovery keys.

Coming on Day 2: dynamic secrets, leases, AppRole and Kubernetes auth, and policy design in depth.